MediSource AI

An Agentic AI-Powered Procurement Automation Platform
for Hospital & Health-System Supply Chains

PROJECT PROPOSAL

Adapted from the FCRIT Projects '26 Teamwise Plan — Team A (Document & Vendor Intelligence), Team B (Approval & Inventory Intelligence), Team C (Contract & Risk Intelligence) and a minimal Team D (Platform & Infrastructure) — re-specialized end to end for the **healthcare procurement domain**.

Domain specialization: Hospital / Health-System Procurement Supply Chain

1. Executive Summary

Hospitals and health systems run procurement operations that are uniquely unforgiving of the inefficiencies that plague generic enterprise procurement: a stock-out of a critical drug or an expired cold-chain vaccine batch is not merely a cost overrun, it is a patient-safety incident. Yet most hospital procurement today still runs on manual document handling, email-based approvals, spreadsheet inventory tracking, and paper contract cycles with biomedical-equipment vendors and pharmaceutical distributors.

MediSource AI proposes an agentic, AI-driven procurement automation platform purpose-built for this domain. It automates the complete source-to-pay lifecycle for hospital procurement — from an incoming supplier invoice to a signed, risk-assessed vendor contract — using intelligent document processing (OCR + extraction), rule-driven and workflow-orchestrated approvals, cold-chain-aware inventory reservation, templated contract generation with e-signature, and a machine-learning vendor risk model tuned to healthcare-specific risk factors (drug-license validity, cold-chain violations, delivery reliability for critical supplies).

The system is designed and delivered as three integrated application teams — **Document & Vendor Intelligence**, **Approval & Inventory Intelligence**, and **Contract & Risk Intelligence** — unified into a single deployable platform, backed by a minimal but real platform/infrastructure layer (containerized services, an event backbone, and a one-command deployment).

2. Problem Statement

Hospital procurement teams face challenges that compound the general problems named in the source plan, with healthcare-specific consequences:

Challenge	Generic Procurement Impact	Healthcare-Specific Consequence
Manual document processing	Slow, error-prone data entry	Misread drug batch/expiry data risks dispensing expired stock
Inefficient vendor onboarding	Long onboarding, fraud exposure	Unverified drug-license/FDA-CDSCO registration risks non-compliant suppliers
Delayed procurement requests	Missed savings, slow cycles	Delayed restock of critical consumables/PPE during demand spikes
Complex, email-based approvals	Bottlenecks, no SLA visibility	Emergency-equipment purchases stall in unstructured approval chains
Limited risk visibility	Reactive risk management	Undetected cold-chain or delivery-reliability failures reach patient care
Contract creation delays	Legal/administrative bottlenecks	Biomedical equipment AMC lapses leave critical devices unmaintained

Challenge	Generic Procurement Impact	Healthcare-Specific Consequence
Manual signature collection	Slow execution, weak audit trail	No verifiable audit trail for regulatory/accreditation review (e.g. NABH/JCI)
No end-to-end visibility	Slow, undocumented decisions	Procurement leadership cannot see exposure across drug/equipment supply chain

3. Objectives

1. Automate ingestion, OCR extraction, and validation of hospital procurement documents (supplier invoices, purchase orders, quotations, drug-license/compliance certificates).
2. Digitize and accelerate medical-vendor onboarding with healthcare-specific KYC (drug-license number and expiry, FDA/CDSCO-style registration ID, GST, sanctions/watchlist screening).
3. Replace email-driven approvals with a policy-as-code, SLA-monitored, auto-escalating approval workflow spanning department, spend, and vendor-risk dimensions.
4. Provide real-time ward/pharmacy inventory visibility with expiry- and cold-chain-aware stock reservation, backorder detection, and reorder suggestions.
5. Auto-generate and route healthcare-specific contracts (Supply Agreement, Biomedical Equipment AMC, Vendor Agreement, NDA) for e-signature, with version/renewal/expiry tracking.
6. Continuously score vendor risk across five dimensions — compliance, fraud, financial, supplier, and operational — using a trained ML model, surfaced before contract execution.
7. Deliver the platform as a single, integrated, independently-deployable system rather than four disconnected team prototypes.

4. Scope

4.1 In Scope

- Team A — Document & Vendor Intelligence: multi-source document ingestion, OCR (PaddleOCR) + OpenCV preprocessing, document classification, field extraction, confidence scoring, duplicate detection, manual-review queue, vendor onboarding & profile management.
- Team B — Approval & Inventory Intelligence: policy-driven (OPA) approval routing, Temporal-orchestrated SLA tracking and escalation, ward/pharmacy stock reservation (Redis-locked), expiry/cold-chain-aware reorder and backorder suggestions.
- Team C — Contract & Risk Intelligence: templated contract generation, OpenSign-compatible e-signature integration, contract lifecycle tracking, scikit-learn vendor risk scoring with MLflow experiment tracking.
- Team D (minimal) — Platform: Docker Compose for local/demo deployment, Kubernetes manifests for a production-shaped deployment, a Kafka event backbone connecting all six services, an Nginx API gateway, database-per-service PostgreSQL, and one-command deploy scripts with health checks and seed data.

4.2 Out of Scope (this iteration)

- Apache Spark distributed processing — deferred as a future scale-out path; single-node OCR/extraction is sufficient at the demo/pilot data volumes this proposal targets.

- Label Studio — an offline OCR/extraction model-quality tool, not a runtime dependency of the deployed platform.
- Full enterprise observability/CI-CD/IaC stack (Prometheus/Grafana/Loki, GitLab CE or Jenkins pipelines, OpenBao secrets management, Terraform/Ansible) — the platform ships with a minimal but real deploy layer (compose + Kubernetes manifests + migrations + health checks) rather than a full production observability suite.
- Authentication/authorization (OAuth2/OIDC) — flagged as a required addition before any real deployment; out of scope for the functional proof-of-concept.
- Integration with a live hospital ERP/EHR system — the platform is built to be integration-ready (documented REST APIs, Kafka events) but no live ERP/EHR connector is implemented in this phase.

5. Team-wise Responsibilities (Healthcare-Specialized)

Team A — Document & Vendor Intelligence

Ingests procurement documents from upload, email, API, and shared-folder sources; runs PaddleOCR + OpenCV preprocessing; classifies documents as invoices, purchase orders, quotations, or compliance certificates; extracts vendor details, line items (including drug batch numbers and expiry dates), taxes, and totals; scores extraction confidence; flags duplicates; routes low-confidence fields to a manual-review UI. Separately manages medical-vendor onboarding: drug-license number/expiry validation, FDA/CDSCO-style registration ID checks, GST validation, and a sanctions/watchlist screening stub, producing an approve/reject/manual-review outcome per vendor.

Team B — Approval & Inventory Intelligence

Defines and evaluates approval rules as code (Open Policy Agent) — spend threshold, requesting department, and vendor risk band determine the approver chain. A Temporal workflow tracks each approval step's SLA deadline, sends reminders, and escalates to the next approver automatically on timeout. In parallel, checks ward/pharmacy stock availability, reserves units against a procurement request with a Redis lock to prevent double-reservation, releases expired reservations, and raises backorder/reorder suggestions for shortfalls — with cold-chain items never split across incompatible batches.

Team C — Contract & Risk Intelligence

Maintains reusable, Jinja2-templated contracts (Supply Agreement, Biomedical Equipment AMC, Vendor Agreement, NDA), auto-populates them from approved procurement and vendor data, and routes the populated contract through an OpenSign-compatible e-signature flow, tracking version history, renewal reminders, and expiry dates. Independently, scores each vendor across five risk dimensions — compliance, fraud, financial, supplier, and operational — with a scikit-learn model whose training runs are tracked in MLflow; high-risk outcomes are surfaced to the contract manager before execution.

Team D (minimal) — Platform & Infrastructure

Provides the shared backbone every other team's service runs on: containerized services (Docker), a Kafka event bus connecting document, vendor, approval, inventory, contract, and risk services without tight coupling, database-per-service PostgreSQL, Redis for locks/deduplication, MinIO for

document/contract object storage, an Nginx API gateway unifying all six services and the frontend behind one origin, and both a Docker Compose stack (local/demo) and Kubernetes manifests (production-shaped) with a single deploy script that runs migrations and seed data.

6. System Architecture

The platform follows a database-per-service microservice architecture. Each of the six services owns a private PostgreSQL database and exposes a FastAPI REST API for synchronous UI-driven interactions. Cross-service side effects — an approved procurement request triggering contract generation and inventory reservation, for example — flow asynchronously through Kafka events, keeping services independently deployable and avoiding direct service-to-service coupling.

6.1 Component Diagram (textual)

React + Bootstrap SPA → Nginx API Gateway → six FastAPI services (document_intelligence, vendor_intelligence, approval_workflow, inventory_intelligence, contract_intelligence, risk_intelligence). All services share: Kafka (event backbone), Redis (locks/dedup), MinIO (document/contract storage), Open Policy Agent (approval + compliance rules), Temporal (long-running approval workflows), and MLflow (risk-model tracking). See **ARCHITECTURE.md** in the repository for the full diagram and per-service detail.

6.2 End-to-End Flow (worked example)

1. A pharmaceutical distributor's invoice is ingested. Document Intelligence stores it in MinIO, runs OCR, classifies it as an *Invoice*, extracts vendor details, line items (drug batch/expiry), taxes, and total; low-confidence fields queue for manual review.
2. The extracted data creates a procurement request. Approval Workflow evaluates OPA routing rules and starts a Temporal-tracked, SLA-bound approval chain.
3. Inventory Intelligence checks ward/pharmacy stock, reserves available units, and raises a backorder/reorder suggestion for any shortfall, respecting cold-chain batch integrity.
4. On approval, Contract Intelligence selects a Supply Agreement or AMC template, populates it, and routes it for e-signature, tracking version/renewal/expiry.
5. In parallel, Risk Intelligence scores the vendor across five risk dimensions; high-risk results are surfaced to the contract manager before final execution.

7. Technology Stack

Layer	Technology	Purpose
Document Processing	PaddleOCR, OpenCV, PyMuPDF	OCR, image preprocessing, PDF rendering
Backend APIs	FastAPI (Python)	All six microservices
Approval & Compliance Rules	Open Policy Agent (Rego)	Policy-as-code for routing & compliance
Workflow Orchestration	Temporal	SLA tracking, reminders, escalation
Event Backbone	Apache Kafka	Cross-service domain events
Relational Storage	PostgreSQL (database-per-service)	Transactional data per service

Layer	Technology	Purpose
Caching / Locks	Redis	Reservation locks, dedup
Object Storage	MinIO	Documents & contracts
Risk Modeling	scikit-learn, MLflow	Vendor risk scoring & experiment tracking
E-Signature	OpenSign (self-hosted)	Contract signature workflow
Frontend	React, Bootstrap	Unified operator UI
API Gateway	Nginx	Single-origin routing to all services
Containerization	Docker / Docker Compose	Local & demo deployment
Orchestration	Kubernetes	Production-shaped deployment

All tools are free / open-source, consistent with the source plan's cost-efficiency principle.

8. Data Model Summary

Full entity-relationship detail is documented in **SCHEMA.md**. Summary per service:

Service	Core Entities
document_intelligence	documents, extracted_fields, duplicate_flags
vendor_intelligence	vendors, vendor_documents
approval_workflow	procurement_requests, approval_steps, approval_history
inventory_intelligence	stock_items, stock_batches, reservations, backorders
contract_intelligence	contract_templates, contracts, contract_events
risk_intelligence	risk_assessments, risk_features_snapshot

9. Expected Business Outcomes

Metric	Target Improvement
Manual document-processing effort	70-90% reduction
Document processing turnaround	60-80% faster
Approval turnaround time	50-70% faster
Vendor onboarding cycle time	Significantly reduced via automated KYC checks
Stock-out / expired-stock incidents	Reduced via expiry- and cold-chain-aware reordering

Metric	Target Improvement
Contract execution time	Reduced via templated generation + e-signature
Risk visibility	Proactive (pre-execution) instead of reactive

10. Deliverables

1. Business case & problem statement (this document, Sections 1-2).
2. Requirement and scope documentation (this document, Sections 3-5; ARCHITECTURE.md).
3. Schema documentation (SCHEMA.md).
4. Architecture diagrams and technical documentation (ARCHITECTURE.md, README.md).
5. Working source code for all six services, unified frontend, gateway, and policies.
6. Kubernetes manifests and Docker Compose stack for deployment.
7. One-command deploy script with database migrations and demo seed data.
8. User manual and test-case documentation (delivered alongside the implementation).

11. Risks & Mitigations

Risk	Mitigation
OCR accuracy on poor-quality scans	Confidence scoring routes low-confidence fields to manual review rather than silently accepting bad data
PaddleOCR model download requires internet on first run	Documented as a deployment prerequisite; models are cached after first pull
Approval bottlenecks if escalation misconfigured	Temporal workflow makes SLA/escalation state inspectable and testable independent of business logic
Vendor risk model trained on limited data	MLflow tracks every training run so model quality is versioned and auditable, not a black box
No authentication in this phase	Explicitly documented as required before any real deployment; gateway is the designed insertion point for OAuth2/OIDC

12. Conclusion

MediSource AI translates the FCRIT Projects '26 procurement-automation plan into a concrete, working system for a domain where procurement failures carry patient-safety consequences. By unifying Teams A, B, and C into one integrated, event-driven platform with a real (if intentionally minimal) deployment layer, the project delivers a demonstrable, end-to-end proof of agentic AI applied to healthcare supply-chain operations — ready to extend toward live ERP/EHR integration, authentication, and full production observability in a subsequent phase.

Repository docs: README.md, ARCHITECTURE.md, SCHEMA.md